Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II. *doi: 10.18637/jss.v000.i00*

sealedge: Sealed Holdouts and Multiple-Testing Defaults for Crypto Backtests in Python

Hansen Winarto
Information Systems Department, BINUS Online Learning
Bina Nusantara University

Abstract

sealedge is a Python package that implements sealed-holdout evaluation and multiple-testing defaults for strategy backtests, with a focus on cryptocurrency perpetual markets as the application domain. The package ships HMAC-SHA256 sealing of holdout inputs (Krawczyk, Bellare, and Canetti 1997), an explore/commit application programming interface (API) in which exploration never opens the seal, Probabilistic Sharpe Ratio (PSR) and deflated-Sharpe diagnostics (Bailey and de Prado 2012, 2014), Superior Predictive Ability (SPA) testing with dual null constructions (Hansen 2005; White 2000), deterministic walk-forward analysis (WFA) (Pardo 2008), and supporting multiplicity tools including Benjamini–Hochberg false discovery rate (FDR) control (Benjamini and Hochberg 1995). The public distribution name is **sealedge**; the importable package remains **quant_lib**. Three registered perpetual-futures experiments illustrate the public explore path. Paper-grade SPA p-values come from a single replication run with `n_spa_iters = 2000` and include both rejections and non-rejections.

Keywords: backtesting, cryptocurrency, multiple testing, Probabilistic Sharpe Ratio, Superior Predictive Ability, walk-forward analysis, Python, reproducible research.

1. Introduction

sealedge is a Python package for sealed-holdout strategy evaluation and multiple-testing defaults (Winarto 2026; Python Software Foundation 2024). It packages established statistical and research-design tools: HMAC-authenticated holdout seals (Krawczyk *et al.* 1997), Probabilistic and deflated Sharpe diagnostics (Bailey and de Prado 2012, 2014), Superior Predictive Ability (SPA) and reality-check testing (Hansen 2005; White 2000), walk-forward analysis (Pardo 2008), and supporting multiplicity utilities including Benjamini–Hochberg FDR control (Benjamini and Hochberg 1995); together these tools form a single importable stack with

a public explore/commit API. The distribution name is **sealedge**; the importable package remains **quant_lib** for API stability (Section 4). No new Sharpe estimator, SPA theorem, or FDR procedure is proposed. The software contribution is how these methods are wired into defaults, return objects, audit surfaces, and a replication path that does not open the holdout during exploration.

Open backtesting systems such as Backtrader, Zipline / Zipline-Reloaded, vectorbt, and QuantConnect Lean provide strong execution and research engines (Rodriguez 2024; Quantopian Inc. 2020; Polakow 2024; QuantConnect 2024). Research-discipline controls (sealed holdouts, SPA/PSR on the same objects as trades, experiment-level multiplicity bookkeeping) are often left to the user rather than shipped as package defaults (Section 6). **sealedge** targets this gap by making the disciplined path the default API surface, rather than an optional notebook appendix.

Cryptocurrency perpetual futures are the motivating application domain for the registered experiments shipped with the package. Continuous trading, funding cash-flows, gaps, and abrupt volatility regimes make holdout leakage and multiple testing easy to understate in ad-hoc scripts. The domain motivates defaults and illustrations; it is not a claim that the methods apply only to crypto markets.

1.1. Scope relative to methods and related software

Holdout evaluation, WFA (Pardo 2008), PSR and deflated-Sharpe methods (Bailey and de Prado 2012, 2014), SPA / reality-check testing (Hansen 2005; White 2000), stationary bootstrap constructions used inside SPA nulls (Politis and Romano 1994), finite-sample permutation p-value hygiene (Phipson and Smyth 2010), and FDR control (Benjamini and Hochberg 1995) are established. **sealedge** implements them on engine return objects and research-session state: sealed holdout inputs, explore that never breaks the seal, commit as an explicit second step, SPA/PSR reporting, deterministic WFA with recorded trials (including **Optuna**-based search (Akiba, Sano, Yanase, Ohta, and Koyama 2019) and a **Numba** bar engine (Lam, Pitrou, and Seibert 2015)), and journal-level multiplicity context for Bonferroni and FDR bookkeeping. Section 2 reviews the methods; Section 5 maps primary claims and additional statistical tools to modules and APIs.

1.2. Contributions

Concrete surfaces shipped by the package:

1. An HMAC-SHA256 sealed-holdout workflow in which explore never opens the seal and commit re-verifies then breaks it once (Section 5).
2. Integrated PSR and deflated-Sharpe diagnostics on the same trade and return objects produced by the engine, including weighted effective sample size constructions (Kish 1965).
3. SPA testing with dual-null software options, Phipson–Smyth p-value formation, and paper-grade defaults (`n_spa_iters = 2000` in the canonical replication path).
4. Deterministic, seedable walk-forward analysis as an engineered realization of established WFA practice, with recorded trials and purge policy.

5. Supporting multiplicity and process tools, including Benjamini–Hochberg FDR, journal Bonferroni context, bootstrap helpers, hypothesis pre-registration fields, and a public `tools.stats` facade, shipped beside the four primary claims (Section 5).

6. An empirical illustration on three registered perpetual-futures experiments that exercises the public explore path and reports mixed SPA outcomes from the paper-grade replication sample (Section 7, Table 2): `vol_compression_v1`, `pullback_sniper_rsi`, and `funding_rate_carry`.

2. Background

Background material below is limited to methods the package implements; packaging into defaults, APIs, and audit surfaces is deferred to Section 5.

2.1. Holdout evaluation and the peeking problem

A standard way to limit overfit is to reserve a terminal holdout window, fit or select on earlier data only, and evaluate the holdout once. In trading research the same idea appears as an out-of-sample block after an in-sample optimization period. The difficulty is operational rather than conceptual: analysts can still look at the reserved window while tuning, silently extend features that depend on post-sample information, or re-run selection until the holdout "works."

Cryptographic sealing does not invent a new validation design. It applies standard hash-and-authenticate primitives (here HMAC-SHA256 over holdout OHLCV and related warmup ranges) so that any modification of sealed inputs is detectable and so that the default exploratory path never opens the reserved window. The methodological content remains ordinary holdout evaluation; the enforcement problem is what the software targets.

2.2. Probabilistic Sharpe Ratio and selection-aware performance

The Sharpe ratio remains a common scalar summary of a return path, but it is sensitive to non-normality and to selection across many trials. Bailey and de Prado (2012, 2014) discuss Probabilistic Sharpe Ratio (PSR) and deflated-Sharpe style corrections that account for estimation error, skewness and kurtosis, and the inflation that arises when the reported strategy is the winner of a search. Related practice also tracks effective sample size under dependence and applies multiple-comparison adjustments when many symbols or trials are scored together.

sealedge treats these diagnostics as implementations to ship, not as notebook formulas reviewers must re-derive. Background for the paper is therefore the published PSR / deflated-Sharpe literature, not a new performance theory.

2.3. Data snooping, reality check, and SPA

When many forecasts or trading rules are compared on the same history, conventional t-tests on the winner are anti-conservative. White (2000) formalizes a reality-check perspective on data snooping; Hansen (2005) develops the Superior Predictive Ability (SPA) test, including

recentering and studentization choices that improve behavior relative to naive benchmarks when many poor alternatives are present.

Finite-sample Monte Carlo or permutation p-values raise a further practical issue: a raw count of extremes can produce a reported p-value of exactly zero. Phipson and Smyth (2010) show why permutation p-values should not be zero when permutations are randomly drawn and recommend add-one style corrections. **sealedge** uses that finite-sample hygiene in its SPA reporting paths.

Across many registered experiments or many simultaneous p-values, the package also ships Benjamini–Hochberg FDR correction (Benjamini and Hochberg 1995) and journal-level Bonferroni context; these multiplicity tools are secondary to the four primary claims and are detailed with the additional statistical surfaces in Section 5.

Two points matter for reading the empirical illustration. First, SPA answers a specific null about superior predictive ability under a chosen resampling construction; it is not a substitute for economic interpretation of costs, turnover, or capacity. Second, non-rejection under the configured null is a possible SPA result. The paper-grade sample therefore reports both rejections and non-rejections without treating either as a software defect.

2.4. Walk-forward analysis

Walk-forward analysis (WFA) optimizes a strategy on a rolling or anchored in-sample window, evaluates the next out-of-sample segment, then advances the window (Pardo 2008). Purge or embargo gaps are commonly inserted so that features with long lookbacks do not leak across the in-sample / out-of-sample boundary. WFA is a research protocol, not a single closed-form estimator: implementations differ in optimizer, objective, parameter freezing rules, and how multi-asset portfolios rebalance risk across folds.

Here WFA is background practice that the package realizes deterministically (fixed seeds, recorded trials, adaptive purge policy). Claims about novelty attach to the engineering of that realization and to its integration with sealed holdout and SPA/PSR reporting (Sections 3–5), not to inventing walk-forward testing as a statistical method.

3. Design principles

The package is organized around a small set of product rules enforced by the public API and session state machine, not optional style advice for notebook users.

When a `ResearchSession` is constructed, holdout-range inputs are hashed and bound under HMAC-SHA256 (Section 5.1). Opening the holdout is not a configuration flag on explore; it is a subsequent `run_commit()` operation that re-verifies the seal and records a one-shot break.

`run_explore()` and the default replication script never break the seal and never return holdout PSR. `run_commit()` requires stage `ready`, re-verifies hashes against cached payloads, opens the holdout once, and only then scores commit-path diagnostics. Reviewers can re-run explore without consuming the holdout.

Walk-forward optimization uses a fixed global seed, per-fold seed algebra, and recorded trial configuration so that a named experiment plus seed reconstructs the same search path (Section 5.4). Purge gaps and per-symbol Optuna searches are part of that engineered path, not

ad-hoc notebook state.

Costs and rejections are exposed. Execution metrics report executed versus rejected trades and costed equity paths. A large signal fire-rate is visible in trade counts and rejections, not hidden behind a single Sharpe number. SPA and PSR consume those costed objects, so friction is inside the statistic rather than a post-hoc footnote.

The canonical replication path fixes `n_spa_iters = 2000` and does not ship a reduced-iteration manuscript script. The paper-grade sample retains whatever SPA outcomes the registered experiments produce under that budget, including non-rejection.

Strategies enter the system as registered hypothesis objects (mechanism, boundaries, success language, period, universe). The candidate state machine

$$\texttt{hypothesis} \rightarrow \texttt{universe} \rightarrow \texttt{edge} \rightarrow \texttt{narrowed} \rightarrow \texttt{ready}$$

blocks invalid backward transitions and stage skips so the state machine blocks exploratory mutation of a frozen claim. Commit is only legal from `ready`.

4. Software architecture

The distribution name on package indexes is **sealedge** (Winarto 2026). The importable Python package name remains **quant_lib** for API stability:

```
import quant_lib
from quant_lib import run_explore, run_commit
```

The command-line entry point is `quant_exp`. Manuscript text uses **sealedge** for the product and **quant_lib** for importable modules.

Modules form a one-way dependency stack from experiment configuration to the Numba engine (names are package paths under **quant_lib**):

```
experiments/  -> cli/  -> research/ -> audit/  -> tools/ -> core/  -> utils/
  registry     Typer    session      seal       API       engine    shared
```

- `experiments/`: named strategy registrations (period, universe, hypothesis metadata, search spaces).
- `cli/`: `quant_exp` commands for list, explore, commit, and reporting without a custom driver script.
- `research/`: session lifecycle, explore pipeline, candidate stages, commit path, result objects, optional plotting.
- `audit/`: holdout seal types, hypothesis factories, experiment journal multiplicity bookkeeping.
- `tools/`: public research helpers, including the `stats` facade for SPA/PSR/FDR.

```
registered experiment (period, universe, hypothesis)
  -> ResearchSession          # HMAC seal created/loaded; closed
  -> Candidate stages
       hypothesis -> universe -> edge (WFA + dual-null SPA)
       -> narrowed / mark_ready()
  -> ExploreResult            # run_explore stops here (seal closed)
  -> run_commit               # re-verify, one-shot break (optional)
  -> CommitResult             # holdout PSR / equity (not in SPA sample)
```

Figure 1: Public explore/commit workflow. Paper SPA tables use `run_explore()` only. Claim 1 seal lifecycle is checked by the synthetic micro-demo in Section 8, not by opening registered-experiment holdouts.

- `core/`: features, WFA, SPA, metrics, and the **Numba** `@njit` bar engine (Lam *et al.* 2015) under costs and constraints.
- `utils/`: shared primitives without pulling research state.

A typical explore call constructs a `ResearchSession` (holdout seal created or loaded; explore may skip holdout feature materialization), builds a `Candidate` from the registered experiment, then runs stages in order:

1. `hypothesis`: bind mechanism, boundaries, and success criteria.
2. `universe`: point-in-time eligibility under age and liquidity filters at the configured train start.
3. `edge`: per-symbol WFA, OOS trade emission, portfolio simulation, dual-null SPA (Section 5.3).
4. `narrowed` / `ready`: optional narrowing then explicit `mark_ready()` before commit is allowed.

`run_explore()` stops at explore metrics with the seal closed. `run_commit()` re-enters from a ready candidate, verifies the seal, breaks once, and scores holdout fields. Result dicts expose commensurate keys (`spa_p_value`, trade counts, equity) so CLI, Python, and replication scripts share one contract.

Most reviewers and users touch three entry points: (i) registered experiment names such as `vol_compression_v1`; (ii) `run_explore()` / `run_commit()` from **quant_lib**; (iii) `quant_exp` for the same flows from a shell. Figure 1 summarizes the sealed path. The replication script calls the explore surface only. Lower layers remain importable for extension, but paper claims are stated against the public explore/commit contract, not against private core symbols.

4.1. Using the software and documentation

A typical research session uses three layers of documentation and one sealed workflow. The product name is **sealedge**; imports and CLI still use **quant_lib** and `quant_exp`.

Formatted help for the library system is provided as NumPy-style docstrings on public modules and functions, browsable API pages under the repository `docs/` tree (built with **mkdocs**; `mkdocs build -strict`), and the narrative methodology write-up used by contributors. After install, interactive help is available via the standard Python help system, e.g. `help(quant_lib.run_explore)` and module-level `help(quant_lib)`. The README states the dual distribution/import names, the explore/commit contract, and the SPA dual-null options so that reviewers do not have to reverse-engineer private core symbols.

Registered experiments are listed from the shell or discovered as named callables. Explore never opens the holdout. The public return type is `ExploreResult` (attribute access and dict-style keys share one field set):

```
# shell: quant_exp list
# shell: quant_exp show vol_compression_v1
from quant_lib import run_explore

result = run_explore(
    "vol_compression_v1",
    cache_dir="./data_cache",  # absolute path preferred
    n_spa_iters=2000,          # paper-grade default
)
# ExploreResult fields (seal stays closed):
result.spa_p_value            # Hansen-path explore p (table column)
result.spa_naive_p_value      # legacy null; may be NaN (span guard)
result.n_oos_trades, result.n_executed, result.n_rejected
result.final_equity
# equivalent: result["spa_p_value"], ...
# shell: quant_exp explore vol_compression_v1
# shell: quant_exp status   # seal still closed after explore
```

The paper-grade replication script calls this explore surface only (Section 8). The keys above match the committed sample used in Section 7. Large SPA p-values are valid outputs; holdout PSR is not on this object.

Holdout evaluation and commit-time diagnostics use an explicit second step. The return type is `CommitResult` (fields include `psr`, `psr_ess`, and holdout equity). The call is irreversible on a live registered seal:

```
from quant_lib import run_commit
# irreversible on a registered experiment seal:
# commit = run_commit("vol_compression_v1", cache_dir="./data_cache")
# commit.psr, commit.psr_ess, commit.final_equity
```

Default manuscript replication does not execute `run_commit()`, so reviewers can re-run explore without consuming the holdout. Claim 1 on the paper path uses the synthetic seal micro-demo instead (Section 8). A mismatch on re-hash at commit raises rather than silently scoring a tampered cache.

Private `core/` symbols are importable for extension but are outside the paper's public contract. The software does not auto-fetch market data into `data_cache/`; missing history fails pre-flight in the replication script rather than inventing bars.

5. Core capabilities

This section maps package statistical surfaces to implementations in **quant_lib**. Subsections 5.1–5.4 cover the four primary claims (method, construction, modules, defaults, explore/commit wiring). Subsection 5.5 covers supporting multiplicity, process, and risk tools at medium depth.

5.1. HMAC sealed holdout

Terminal holdout evaluation reserves a window for a single final score after selection on earlier data. The statistical design is ordinary holdout evaluation; the software problem is integrity and process control. Reserved OHLCV and any feature inputs that can change signals must not be silently rewritten between session creation and final scoring, and the default exploratory path must not score the reserved window at all. The package authenticates seal records with HMAC-SHA256 (Krawczyk *et al.* 1997).

On `ResearchSession` construction the package loads holdout-range inputs from the data cache and computes a SHA256 digest. The hasher lives at `_compute_holdout_data_hash` in the session module (see the module-path lists at the start of each tool description and in Section 5). Let $\mathcal{S}$ be the sorted symbol set. For each $s \in \mathcal{S}$ the hasher feeds a CSV serialization of every present column (open, high, low, close, volume) with no row index. Hashing the full OHLCV row rather than just close detects tampering on path-dependent stops or volume-driven features. Extended-history frames used for EMA warmup at commit time are hashed under the same column rule. If funding series are present, each symbol contributes `time` and `funding_rate` columns because funding enters trade PnL in the engine. The digest is therefore a fingerprint of every input that can change holdout scores under the package's feature and cost model.

Seal record and authentication. The digest is written into a seal object (`HoldoutSet` in `audit/holdout.py`) with holdout start and end dates, a UTC sealed-at timestamp, an optional broken-at time, and an HMAC-SHA256 signature. The seal file is JSON. The MAC uses a canonical serialization with sorted keys and tight separators, excluding the signature field and the local path so that moving the file does not invalidate an otherwise intact seal. The secret is read only from `QUANT_LIB_HMAC_SECRET` (minimum 32 characters); there is no insecure default. Verification uses constant-time comparison (`hmac.compare_digest`). Post-signature checks match the stored data hash and detect a prior break.

Workflow invariants.

1. `run_explore()` builds the session with `_skip_holdout_load=True` so the public explore path never materializes holdout features for scoring; the seal is not broken.
2. `run_commit()` requires candidate stage `ready`, checks `is_sealed()` and `verify()`, recomputes the SHA256 from the session's cached holdout, BTC-extended, and funding payloads, and aborts on mismatch before any holdout trades are scored.
3. `commit_break` re-verifies disk state, then writes `broken_at`, the post-break `data_hash`, and a fresh HMAC in a single save. A broken seal cannot be re-sealed; a later session

against the same seal path loads the prior broken state when the signature remains valid under the current secret.

The cryptographic primitives are standard. What is package-specific is session-level wiring: hash coverage, one-shot break, and the explore/commit split that keeps paper-grade SPA replication explore-only. Audit-process instruments (Subsection 5.5.4) pre-date the seal and supply the registered experiment bindings that anchor each `HoldoutSet` to a named strategy.

Replication micro-demo (Claim 1). The SPA path under `replication/output_paper_grade/` never opens a registered-experiment holdout. A separate synthetic script exercises Claim 1 for reviewers: `scripts/reproduce_seal_demo.py`, runnable via `make reproduce-seal`.

The script seals a temp JSON holdout, then verifies it. It tampers the on-disk `data_hash` so `verify()` fails. It runs `commit_break` once and checks one-shot invariants (second break fails, reseal raises). The committed cross-check sample lives under `replication/output_seal_demo/`. That path does not score market trades and does not call `run_commit()` on paper experiments.

5.2. Probabilistic Sharpe Ratio on engine returns

Bailey and de Prado (2012, 2014) define and develop the Probabilistic Sharpe Ratio as the probability that the true Sharpe ratio exceeds a benchmark after adjusting for estimation error under non-normal returns. With sample Sharpe $\widehat{\mathrm{SR}}$, benchmark SR^*, sample size n (or effective sample size n_{eff}), skewness γ_3, and regular kurtosis γ_4,

$$\widehat{\mathrm{PSR}} = \Phi\left(\frac{\widehat{\mathrm{SR}} - \mathrm{SR}^*}{\sqrt{\widehat{V}/(n_{\mathrm{eff}} - 1)}}\right), \quad \widehat{V} = 1 - \gamma_3\,\widehat{\mathrm{SR}} + \frac{\gamma_4 - 1}{4}\,\widehat{\mathrm{SR}}^2. \tag{1}$$

With excess kurtosis $\kappa = \gamma_4 - 3$ (Fisher convention as in **scipy.stats.kurtosis**), the kurtosis term equals $(\kappa + 2)/4$. The same variance-of-SR structure appears in the deflated-Sharpe construction discussed in Subsection 5.5.

Function `prob_sharpe_ratio` in `quant_lib/core/_testing.py` implements that formula on a one-dimensional return array.

- Unweighted path: sample mean and sample standard deviation (`ddof=1`), denominator $n - 1$, return $(\mathrm{NaN}, \mathrm{NaN})$ if $n < 10$, zero variance, or non-finite extreme SR.
- Weighted path: weighted mean and variance after normalizing weights to sum to one. Kish ESS (Kish 1965) $n_{\mathrm{eff}} = 1/\sum_i w_i^2$; reject if ESS < 2.
- Optional annualization multiplies SR (and rescales the benchmark) by $\sqrt{365.25}$ for daily series. Commit-time trade PSR disables annualization because inputs are per-trade R-multiples, not daily returns.
- Variance correction is clipped to a small positive floor when Bailey's asymptotic expansion leaves the valid range (very high SR with near-normal higher moments), so callers receive a usable PSR near one rather than a silent NaN from a negative variance term.

Skewness and excess kurtosis are computed with **scipy.stats**. The WFA in-sample objective reuses the same Bailey variance correction so search and reporting share one formula family (Subsection 5.4).

Explore versus commit. `run_explore()` never opens the seal and therefore does not return the holdout PSR. Paper-grade `scripts/reproduce.py` is explore-only and reports SPA and trade counts. Holdout PSR is computed in `quant_lib/research/commit.py` on executed holdout R-multiples after seal break. Reviewers comparing manuscript tables to the public explore path should not expect a holdout PSR column.

Relation to supporting tools. The PSR surface reuses the Kish ESS gate from Subsection 5.5.3 and shares its variance-of-SR term with the deflated Sharpe computation in Subsection 5.5.2. Both pieces consume the same `prob_sharpe_ratio` return tuple so that commit-time PSR and deflated Sharpe never disagree on higher moments of the trade series.

5.3. SPA with dual null constructions

Hansen (2005) SPA tests whether the best of K competing rules has superior predictive ability relative to a benchmark, with studentization and recentering that improve behavior when many poor alternatives are present. White (2000) supplies the related reality-check background. Finite-sample Monte Carlo p-values use the Phipson–Smyth add-one form

$$p = \frac{1 + \#\{\text{null} \geq \text{obs}\}}{B + 1} \tag{2}$$

so p cannot be zero under random draws (Phipson and Smyth 2010).

Shared entry point. `portfolio_spa` in `quant_lib/core/_spa.py` is the single SPA entry. Callers pass observed out-of-sample trade dicts (including `r_net`, times, symbol, and stop parameters), per-symbol asset panels, daily close and high/low matrices, portfolio risk knobs, and iteration budget B (`n_iters`; paper default $B = 2000$ via `run_explore()`). Two null constructions coexist. The public edge path evaluates both and stores both p-values.

Legacy null (always computed). The legacy null preserves cross-asset co-occurrence by time-anchored circular permutation of the observed trade set:

1. Simulate observed portfolio equity E_{obs} with `simulate_full_portfolio` under the same leverage, margin, position-limit, and circuit-breaker settings as the live path.
2. Record each trade's hour offset from the first entry and its duration.
3. For $b = 1, \ldots, B$, draw a uniform anchor in the global hour span, remap every trade entry by modular arithmetic on the symbol timeline, re-simulate exits under the same WFA cost model (fees, weekend penalty, stress multiplier), then re-run the portfolio simulator to obtain E_b. Exit re-simulation uses the package trailing-stop helper.
4. Set $p_{\text{naive}} = (1 + \#\{E_b \geq E_{\text{obs}}\})/(B + 1)$.

Degenerate guards return non-misleading sentinels: empty trade list → $p = 1$; NaN observed equity → p = NaN; all null equities stuck at initial capital → $p = 1$; anchor span at least 80% of the calendar → legacy p = NaN (null nearly identical to the observed path). In the paper-grade explore sample under `replication/output_paper_grade/`, the stored `spa_naive_p_value` fields are NaN for all three strategies under that anchor-span guard, while `spa_p_value` remains the Hansen-literal p (or Hansen's fallback to the legacy p when Hansen cannot run). Table 2 therefore reports Hansen-path `spa_p_value` only; a NaN legacy field is an intentional guard output, not a failed SPA run. Legacy p is finite when the remapped trade anchor span is under the 80% calendar threshold—typically shorter evaluation windows, lighter lookbacks, or denser bars relative to trade duration—so dual storage is not redundant outside this sample. Explore defaults still treat Hansen-path `spa_p_value` as the primary reported column.

Hansen-literal null (explore default). When the recenter policy is Hansen-literal, statistics are returned, and WFA supplies non-empty per-trial in-sample PnL arrays (`trial_r_nets`), a NumPy-only Hansen helper runs without calling the trade simulator. Legacy permutation RNG streams stay untouched. Hansen uses a separate generator seeded at `seed+1`. Under a zero-mean benchmark, loss differentials are $d_k = -r_{\text{net},k}$. For each trial k and bootstrap draw b, stationary block resampling of d_k (Politis and Romano 1994) with expected block length $p_k = \max(1, \text{round}(n_k^{1/3}))$ (or a static override) yields studentized statistics

$$T_b^k = \sqrt{n_k}\, \bar{d}_b^k / \widehat{\sigma}(d_k).$$

Hansen Equation 7 recentering discards negative-mean bootstrap statistics; Equation 8 takes the cross-trial maximum $T_b^{\text{null}} = \max_k T_b^{k,\text{acc}}$. The observed statistic uses the out-of-sample winner's r_{net} the same way. Then

$$p_{\text{hansen}} = (1 + \#\{T_b^{\text{null}} \geq T_{\text{obs}}\})/(B + 1).$$

On missing trials, $N_{\text{obs}} < 2$, or zero-variance series, Hansen falls back to the legacy p and records `fallback=True`.

`Candidate.run_edge_testing` aggregates `trial_r_nets` from every WFA fold, calls `portfolio_spa` with Hansen kwargs, and stores

- `spa_p_value` = p_{hansen} (or legacy on fallback);
- `spa_naive_p_value` = p_{naive}.

Paper Table 2 reports the explore-path `spa_p_value` at $B = 2000$. The public thin wrapper `tools.stats.spa_test` exposes the legacy three-tuple API; paper-grade numbers come from the explore pipeline's Hansen settings, not from inventing a different null in that wrapper.

Relation to supporting tools. The SPA null construction runs against the same portfolio equity path that the cost accounting layer (Subsection 5.5.5) builds, so a reviewer auditing the legacy permutation can trace every per-trade cost component from engine to null. Across the registered-experiment family, the Bonferroni / journal counter (Subsection 5.5.1) reports n_{tests} and α_{adj} beside the SPA p so the explore table can be read alongside its family-level multiplicity context. The FDR step-up (Subsection 5.5.1) is intentionally not applied to the paper Table because the three SPA p-values are a fixed, pre-registered family and applying FDR would only rescale them toward one without changing the rejection set.

Hansen-fallback diagnostics. When Hansen cannot run, either because `trial_r_nets` is missing, because fewer than two observed trades survive, or because trial or observed series are zero-variance, the explore path records `hansen_fallback = True` on the candidate `edge_metrics` object and returns the legacy p in place of p_{hansen}. `spa_naive_p_value` remains the legacy circular-permutation p in the same return object. In the paper-grade explore sample under `replication/output_paper_grade/`, the published `spa_p_value` columns are Hansen-path values (no fallback substitution into the table), while `spa_naive_p_value` stays NaN for all three strategies under the legacy anchor-span guard. Reviewers can confirm the two SPA p fields from `results.json` without re-running the SPA pipeline; the boolean fallback flag itself is an in-process `edge_metrics` diagnostic rather than a top-level sample key.

Finite-sample divergences and disclosures. Relative to a strict reading of Hansen's asymptotic SPA setup, the Hansen-literal path discloses four finite-sample choices that the implementation does *not* silently "fix": (i) expected block length is the Politis–Romano rule $p = \max(1, \text{round}(n_k^{1/3}))$ per trial (or a fixed override via `STATIC["spa_hansen_block_length_override"]`), without automatic Patton–Politis–White block selection; (ii) trial lengths n_k may differ across Optuna trials, and the path does not apply a $\sqrt{N/n_k}$ sample-size rescale; (iii) Hansen's two-stage q bootstrap (Hansen 2005, Section 3) is omitted—the Eq. 7 recenter plus Eq. 8 max-statistic is the data-snooping correction used here; (iv) finite-sample uniformity of the max-of-K null is an empirical calibration statement at finite B and n_k, not an asymptotic theorem. Two sample-path guards sit beside those divergences and are visible in the paper-grade explore artifacts: the legacy circular permutation returns NaN when the anchor span is at least 80% of the calendar, and the Hansen path substitutes the legacy p with `hansen_fallback = True` on zero-variance or singleton inputs (not triggered for the published table). Both p-values use the Phipson–Smyth add-one form, so a published $p \approx 0.0005$ at $B = 2000$ reflects $\#\{\text{null} \geq \text{obs}\} = 0$, not a zero-cell artefact. Monte Carlo standard error is $\mathcal{O}(1/\sqrt{B})$; paper-grade results fix $B = 2000$ and reviewers should not down-sample.

5.4. Deterministic walk-forward analysis

Walk-forward analysis optimizes on rolling or expanding in-sample windows, evaluates the next out-of-sample block, and advances (Pardo 2008). Purge gaps reduce boundary leakage from long-lookback features. The package does not invent WFA; it engineers a seedable, recorded realization integrated with seal, SPA, and risk allocation.

`run_wfa_per_symbol` in `quant_lib/core/_wfa.py` sorts months per eligible symbol. Folds use expanding in-sample history from `wfa_min_train_months` with out-of-sample length `wfa_test_months`. Adaptive purge days shrink with longer in-sample history (90 / 60 / 30 days by in-sample month buckets) via `_get_purge_days`. Folds with too few bars or gaps larger than `max_allowed_gap_hours` are skipped. The per-fold seed is

$$\texttt{fold\_seed} = \texttt{GLOBAL\_SEED} \oplus \sum_{c} \text{ord}(c) \oplus (y \cdot 100 + m),$$

where the sum is over characters of the symbol ticker and (y, m) is the last in-sample month. Independent RNG streams for Optuna warm-start perturbation, out-of-sample cost noise, and in-sample cost noise use fixed XOR masks on that seed (`0xDEF`, `0xABCD`, `0xBEEF`).

Each fold runs **Optuna** `TPESampler(seed=fold_seed)` (Akiba *et al.* 2019) with an adaptive trial budget. `WalkForwardObjective` suggests strategy parameters from the experiment search space, runs the **Numba** `fast_trade_loop` (Lam *et al.* 2015) on purged in-sample bars, and scores a PSR-style objective with time-decay bar weights (halflife in months), Kish ESS trade reweighting (Kish 1965), optional L2 pull toward parameter centers, and a log-ESS trade-count weight. Early exits (too few bars/trades, non-positive weighted variance) return a large negative score and append a `None` sentinel to `trial_r_nets`; successful trials append the in-sample PnL array consumed later by Hansen SPA.

Winning parameters are frozen per fold; out-of-sample trades are generated with the same engine and fold-specific cost RNG. Fold metadata records best parameters, objective value, and the full `trial_r_nets` list (JSON-friendly lists so reproducibility equality tests remain unambiguous). Across symbols, `Candidate.run_edge_testing` concatenates out-of-sample trades, picks per-symbol frozen parameters by best fold PSR, applies profit-factor-weighted risk allocation preferably from in-sample trades (decoupled from the out-of-sample SPA sample; see Subsection 5.5), simulates the portfolio, then runs SPA as in Claim 3.

Reviewers should check determinism (same experiment name, data, and seeds reconstruct the same fold path), purge policy, and the explicit handoff of in-sample trial PnL into Hansen SPA; not a new theory of walk-forward testing.

Relation to supporting tools. The WFA in-sample objective reuses the Kish ESS gate from Subsection 5.5.3 so that search cannot manufacture precision with degenerate weighting. Across folds, the PF-weighted risk allocator (Subsection 5.5.4) takes the per-fold `trial_r_nets` list as input, runs halflife-decayed PF weighting on the IS trade pool, and returns a final symbol-risk dict that the portfolio simulator resizes. Together with Claim 3, this closes the chain WFA IS trials → Hansen SPA inputs → portfolio equity → costed R-multiples used in Claim 2.

5.5. Additional statistical and process tools

The four primary claims do not exhaust the statistical surface of **quant_lib**. This subsection documents the supporting tools that ship with the package, the formulas they implement, the code paths that wire them to the explore and commit pipelines, and how each tool relates to the primary claims. Eleven supporting tools are grouped into five clusters: multi-test accounting (5.5.1, 5.5.1), selection-inflation control (5.5.2, 5.5.2), path-risk quantification (5.5.3, 5.5.3), audit and process instrumentation (5.5.4, 5.5.4), and cost / engine realism (5.5.5, 5.5.5). Each entry is treated at medium depth, enough for a reviewer to locate, audit, and reproduce the implementation by following the named module. Source-of-truth module paths are inline at the start of each tool's description.

Multi-test accounting

Benjamini–Hochberg FDR correction. Given an unordered vector of raw p-values $p_1, \ldots, p_n$ with significance target α, Benjamini and Hochberg (1995) replace the family-wise Bonferroni bound by the step-up procedure: sort p ascending into $p_{(1)} \leq \ldots \leq p_{(n)}$, find the largest k such that $p_{(k)} \leq k\alpha/n$, and reject all hypotheses with $p \leq p_{(k)}$. Ad-

justed q-values follow as $q_{(i)} = \min_{j \geq i} p_{(j)} j/n$ (right-to-left cumulative minimum) so monotone non-decreasingness is preserved. Function `fdr_correction` in `quant_lib/core/_testing.py` returns the tuple `(reject_mask, q_values)`; the CLI alias `fdr_correct` in `quant_lib/tools/stats.py` re-exports it for notebook and `quant_exp` callers. The result drives Table-level FDR columns only when the table has more than a handful of cells; the paper Table 2 is left raw because the three SPA p-values form a fixed, pre-registered family and FDR would just rescale them toward one with no new inferences.

Bonferroni context and the experiment journal. Multiplicity across an entire research program is the count of registered experiments that have been pushed through the machine, not the row count of one Table. The `ExperimentJournal` class in `quant_lib/audit/journal.py` threads a registry counter across every `run_explore` / `run_commit` call: `log_run(description, category, params_snapshot)` appends a structured entry and `n_tests` returns a credit-weighted count

$$n_{\text{tests}} = n_{\text{experiments}} + 0.5 \cdot n_{\text{ablations}},$$

which feeds the next-test adjusted significance level

$$\alpha_{\text{adj}} = \alpha_0/(n_{\text{tests}} + 1).$$

Bugfix entries are logged but excluded from the counter, so a fix-up round does not inflate multiplicity. The +1 in the denominator avoids division-by-zero before any experiment has been run. The CLI report layer prints Bonferroni and FDR context fields beside SPA so the reviewer sees the multiplicity state without opening the journal file.

Selection-inflation control

Deflated Sharpe Ratio. When the in-sample optimizer is Optuna and the search space contains many trial configurations, the family size N that matters for selection inflation is the Optuna search budget, not the registered-experiment count. Bailey and de Prado (2014) derive the deflated Sharpe probability

$$\text{DSR} = \Phi((\widehat{S} - S_{\max})\sqrt{\frac{n-1}{1 - \gamma_1 \widehat{S} + \frac{\gamma_2 - 1}{4}\widehat{S}^2}}),$$

where $\widehat{S}$ is the observed Sharpe, $S_{\max} \approx \sqrt{2 \ln N} - \frac{\ln(\ln N) + \ln(4\pi)}{2\sqrt{2 \ln N}}$ approximates the expected maximum of N null Sharpes via the normal tail quantile, and γ_1, γ_2 are skewness and excess kurtosis of the trade R-multiple series. Function `deflated_sharpe_ratio` in `quant_lib/core/_testing.py` accepts `(observed_sr, n_trials, skew, excess_kurt, n_observations_per_trial=None)` and returns DSR $\in [0, 1]$; the implementation returns NaN when $N < 2$ (no multiple testing) and 0 when the observed Sharpe does not beat $S_{\max}$. Commit reporting attaches DSR when the family size is known; the explore-only paper Table 2 centers SPA because SPA already wraps N trials into its own null.

P-value labeling. The CLI, HTML report layer, and PDF output all read p-values from the same `ExploreResult` objects. A reviewer cross-checking them should not be told that $p = 0.93$ is "inconclusive" on one page and "non-rejecting" on another. Function `label_p_value` in `quant_lib/core/_testing.py` takes `(p, context)` (where `context` is one of `"mean_r"`, `"spa"`, or `"bonferroni"`) and returns a triple `(category_label, strength_label, prose_label)` keyed at $\alpha = 0.05$: "strong reject" for $p < 0.01$, "reject" for $p < 0.05$, "borderline" for $p < 0.10$, and "non-reject" for $p \geq 0.10$. Numeric p stays byte-identical across PDF, terminal, and notebooks. Only the wording is centralized.

Path-risk quantification

Bootstrap helpers for path risk. Whether the realised R-multiples are statistically superior is one question; whether the same path survives resampling is another. Circular block bootstrap on a trade- or day-return series with expected block length b approximates the sampling distribution of CAGR and maximum drawdown without assuming independence. Function `bootstrap_daily_returns` in `quant_lib/core/_metrics.py` picks a block length under the Politis and Romano (1994) rule $\widehat{b} = (\lfloor 3.15\, n^{1/3} \rfloor, \lceil 1.5\, n^{1/3} \rceil)$ so the polling bias is small relative to the asymptotic-variance estimate; for sparse trade series `bootstrap_trade_multiples` resamples blockwise over the R-multiple list with the same expected block length. Daily bootstrap pads each zero-return day as a no-trade day at block boundaries, which inflates sample size and biases path-risk estimates; trade block bootstrap is the default on `CommitResult` when the number of trades is much smaller than the number of calendar bars. Both helpers reuse the Politis–Romano block primitives already used inside Hansen SPA; they are not substitutes for Claim 3's dual null.

Kish effective sample size. When weights under time-decay (or any other uneven-exposure scheme) concentrate the mass on a few trades, the nominal sample size n overstates precision. Kish (1965) define the effective sample size of a non-negative weight vector w as

$$n_{\text{eff}} = \frac{(\sum_i w_i)^2}{\sum_i w_i^2} = \frac{1}{\sum_i \bar{w}_i^2}, \qquad \bar{w}_i = \frac{w_i}{\sum_j w_j},$$

so uniform weights give $n_{\text{eff}} = n$ and a single mass at one trade collapses to $n_{\text{eff}} = 1$. The package does not expose a separate public `kish_ess_n` helper; Kish ESS is computed inline inside the weighted branch of `prob_sharpe_ratio` in `quant_lib/core/_testing.py` as $n_{\text{eff}} = 1/\sum_i w_i^2$ after weight normalization, and the same ESS gate rejects weighted-PSR formation when $n_{\text{eff}} < 2$. The WFA in-sample objective reuses that weighted-PSR path, so search cannot manufacture precision with artificial weighting and the commit-time PSR surface shares one ESS definition.

Audit and process instrumentation

Hypothesis objects and pre-registration fields. A strategy that aspires to pre-registered evaluation needs declared mechanism, boundary conditions, and success criteria before it sees data. `register_hypothesis` (and the `Hypothesis` dataclass) in `quant_lib/audit/hypothesis.py` puts that pre-registration into the code: it requires

five narrative fields – `name`, `mechanism`, `boundary_conditions`, `success_criteria`, and one of `entry_logic` or `exit_logic` – and rejects blank or generic entries with a typed `HypothesisValidationError`. A `QualityLevel` (`DRAFT`, `REVIEWED`, `REGISTERED`) flags whether the hypothesis is human-reviewed or machine-fast-tracked. `experiments/` modules bind a strategy factory name plus these fields to a registered experiment on disk, so that `run_explore()` can replay the same hypothesis + seed + universe triple that produced the original p-value. This is an audit surface, not a statistical estimator, but it changes the way SPA inputs are interpreted because the success language is pre-committed.

Profit-factor-weighted risk allocation. Across the universe, edge testing builds one portfolio from per-symbol OOS trades. Equal-weight allocation is the naive choice; the package instead uses per-fold Profit Factor (PF) as a cheap meta-weight. `apply_pf_weighted_risk_allocation` (and the IS-decoupled sibling `apply_pf_weighted_risk_allocation_is`) in `quant_lib/core/_risk_allocation.py` proceeds per OOS fold as follows. Past folds contribute decay-weighted gross profit and loss with $d_h(n_{\text{back}}) = 0.5^{n_{\text{back}}/h}$, yielding a per-symbol PF. That PF is mapped to a factor

$$f_j = \text{clamp}(\text{PF}_j,\ f_{\min},\ f_{\max}), \qquad f_{\min} = 0.5,\ f_{\max} = 1.5$$

with those clamp defaults (or 1.0 when past trade count is below a configured minimum; infinite-PF winners take $f_{\max}$). Factors are then scaled by the baseline per-symbol risk and rescaled so the active-universe total risk budget is preserved (the X1 scheme). Explore prefers the IS trade pool for past-fold PF when available, which removes the double-dipping failure mode that would arise if weight selection and SPA null construction shared the same OOS pool. `extract_final_fold_weights` maps the last fold's weights into a final symbol-risk dict that `simulate_full_portfolio` resizes under leverage and margin limits. The portfolio block (`quant_lib/core/_portfolio.py`) treats this dict as a default. It is not a hypothesis test: changing the weights shifts the SPA input measure but does not change the SPA null construction.

Cost accounting and engine realism

Costs, rejections, and engine realism. The **Numba** bar engine in `quant_lib/core/_engine.py` and the portfolio simulator in `quant_lib/core/_portfolio.py` build a single costed object per filled trade. On every fill the engine forms

$$R_{\text{net}} = R_{\text{gross}} - c, \qquad c = \min(c_{\text{raw}},\ 5.0),$$

where c_{raw} reuses the engine's existing cost fields: round-trip taker fees `fee_taker` (= 0.05 percentage points per side, i.e. 0.05% of notional, matching the engine's ATR-percent cost scale), ATR-scaled entry and exit slippage multiplied by $1+U\cdot(\texttt{stress\_test\_multiplier}-1)$ with $U \sim \text{Uniform}(0,1)$ as the random stress draw, the weekend liquidity penalty `weekend_liquidity_penalty` (= 2.0) on weekend entry bars and on exits that held over a weekend bar, and accumulated funding impact on funding-hour bars for carry strategies. The portfolio layer applies maintenance margin, leverage (3× default), a global concurrent-position limit (default 4), and circuit-breaker cooldowns; rejected signals are counted per reason – `position_limit`, `margin_insufficient`, `cb_cooldown`, `invalid_sl_pct` – rather than dropped, so

`n_executed` and `n_rejected` can be compared against `n_oos_trades` in `results.json`. A high signal fire-rate (as in the funding-rate case study, where `n_oos`=3446 produces `n_executed`=1517 and `n_rejected`=1929) is visible immediately and cannot hide behind a single p-value.

Public `tools.stats` facade. Module `quant_lib/tools/stats.py` is a thin re-export layer that ships `spa_test` (legacy 3-tuple SPA), `prob_sharpe_ratio` (PSR), and `fdr_correct` (FDR). The Hansen-literal SPA and the full portfolio SPA kwargs stay on `quant_lib.core._spa.portfolio_spa`, so callers can reach the advanced path from `quant_exp` or notebook without fumbling through `core/` submodules. The facade is consistency-only – it does not modify the numeric tests – but centralising the names keeps `quant_exp report` and the JSS paper Table at the same wording.

The eleven tools above group into five audit clusters. Multi-test accounting (Subsections 5.5.1–5.5.1) sits next to Claim 3 (Subsection 5.3). Selection-inflation control (5.5.2–5.5.2) sits next to Claim 2 (Subsection 5.2). Path-risk quantification (5.5.3–5.5.3) pairs with Claim 4 (Subsection 5.4). Audit instruments (5.5.4–5.5.4) pre-date Claims 1 and 4 – PF weighting feeds the same edge pipeline. Cost accounting (5.5.5–5.5.5) is the realism floor under the SPA inputs used by Claim 3.

Together, these tools make multiplicity, selection-deflation, path-risk bootstrap, journal bookkeeping, and cost accounting available on the package without requiring users to reimplement block-bootstrap or PF-weighting by hand. Every paragraph above points to a specific module path a reviewer can audit; numerical claims in Section 7 remain the explore-path SPA sample under the primary SPA configuration.

6. Comparison with related software

We position **sealedge** against four widely used open stacks: Backtrader (Rodriguez 2024), Zipline / Zipline-Reloaded (Quantopian Inc. 2020), vectorbt (Polakow 2024), and QuantConnect Lean (QuantConnect 2024). The matrix below is intentionally conservative. Cells answer whether the capability is a documented product surface for research discipline (sealed holdout, SPA/PSR pipeline, deterministic WFA records), not whether a motivated user could reimplement the idea on top of a general backtester. The label "user" means achievable by custom scripting but not shipped as a sealed pipeline; "no" means we found no equivalent in project docs at draft time.

Two observations follow. First, competitors are strong execution and signal engines; their gap relative to **sealedge** is not raw backtest speed but statistical-hygiene defaults (cryptographic holdout invariants, dual-null SPA, PSR on the same objects as trades, and journal/FDR multiplicity surfaces). Second, Lean is omitted from the narrow table for width, but the same discipline gap applies: a cloud-capable multi-asset engine without a package-default HMAC seal plus SPA explore path. We do not claim competitors cannot run walk-forward or holdout studies; only that **sealedge**'s contribution is packaging those practices so explore cannot silently peek and so SPA/PSR are not optional notebook afterthoughts.

7. Empirical illustration

Capability	Backtrader	Zipline	vectorbt	**sealedge**
Primary engine style	event	event	vector	hybrid[a]
HMAC sealed holdout workflow	no	no	no	yes
Built-in SPA + PSR pipeline	no	no	no	yes
Documented WFA + trial records	user	user	user[b]	yes
Built-in funding-rate features	no	no	no	yes

Table 1: Comparison of research-discipline capabilities. "user" denotes user-built extensions rather than a sealed, package-default workflow. [a]**sealedge** uses a Numba bar engine with vectorized feature paths. [b]vectorbt documents walk-forward and combinatorial CV patterns (especially in PRO materials); free/open surfaces still leave SPA/PSR and HMAC sealing to the user. Funding-rate features refer to perpetual-funding signals in the experiment stack (e.g., `funding_rate_carry`), not the mere ability to trade crypto symbols in a general engine.

Strategy	OOS trades	Executed	SPA p	Runtime (s)
`vol_compression_v1`	438	399	1.0000	691
`pullback_sniper_rsi`	180	140	0.9300	345
`funding_rate_carry`	3446	1517	0.0005	2138

Table 2: Paper-grade explore metrics from the committed replication sample (seed 42, `n_spa_iters = 2000`). SPA p is `spa_p_value` under the explore Hansen path; mixed outcomes are expected and disclosed. Legacy naive p may be NaN in the same JSON sample (span guard).

This section is an illustration of the public explore path, not a strategy recommendation study. Each registered experiment is run with the seal closed; reviewers re-run the same path via Section 8. Strategy mechanisms are stated only as far as needed to identify the experiment.

We evaluate three registered experiments on perpetual futures for `BTCUSDT`, `ETHUSDT`, and `SOLUSDT`, with training start `2021-07-01` (driven by the latest-starting symbol under universe age filters). SPA uses `n_spa_iters = 2000` and global seed 42. Results below are bit-identical to the committed sample under `replication/output_paper_grade/`. Table cells are explore-path `spa_p_value` (Hansen wiring); legacy `spa_naive_p_value` is NaN in that sample under the span guard of Subsection 5.3 and is not a table column. Seal break is illustrated separately in Subsection 5.1 via the synthetic micro-demo, not by opening these experiments' holdouts.

7.1. Volatility compression (deep illustration)

Registered name `vol_compression_v1`; the same explore path, universe, calendar, seed 42, and `n_spa_iters = 2000` used Section-wide also apply here. The mechanism is a volatility-compression breakout: after low realized volatility (`vol_pct_rank`) a close through the recent 20-bar high or low, with a one-bar pullback confirmation, opens a trade; exits use an ATR-scaled trail with a fixed bailout bar count. Hypothesis object records mechanism, boundaries, and pre-registered success language; period configuration fixes train start `2021-07-01`, train end `2025-12-31`, and a six-month post-training holdout (sealed during explore). The package runs universe filters → per-symbol WFA with adaptive purge and Optuna → OOS trades

under the costed engine → portfolio simulation with IS-preferring risk weights → dual-null SPA and trade counts.

On the committed cross-check sample the run yields 438 OOS trades (399 executed, 39 rejected), terminal equity about 893 on the 1000 notional path, SPA $p = 1.0000$ (Table 2), runtime about 691 s on the sample host. Equity below starting notional and a non-rejecting SPA p mean the strategy does not demonstrate superior predictive ability against the configured null on this window; the case is the deep illustration because the full pipeline completes to a reviewer-checkable artifact, not because the strategy wins. The Claim 1 synthetic seal micro-demo (Section 8) confirms the seal lifecycle on the paper path; this strategy's explore-path seal stays closed.

7.2. Pullback sniper (brief)

`pullback_sniper_rsi` runs the same explore surface with the same calendar, seed, and SPA budget. The mechanism is RSI-extreme mean reversion with reversal-candle entry and trail / take-profit / bailout exits. On the committed sample it emits 180 OOS trades (140 executed), SPA $p \approx 0.9300$ (Table 2); the p-value does not support rejecting the null. Role of this case: a stylistically different rule still flows through the same sealed explore path and returns commensurate metrics, demonstrating the shared experiment registry rather than the rule's edge.

7.3. Funding-rate carry (brief)

`funding_rate_carry` runs the same explore surface and SPA configuration. The mechanism is structural mean reversion in perpetual funding: short when funding percentile rank exceeds an entry threshold, long when it falls below the symmetric low threshold, with exits on return to a neutral band, trail, or bailout. On the committed sample it emits 3446 OOS trades (1517 executed, 1929 rejected), terminal equity about 504, SPA $p \approx 0.0005$ (the only reject among the three; Table 2). Two disclosures: (i) rejecting the SPA null is not an unconditional trading recommendation; costs, rejections, and the equity path still matter; (ii) the large trade count is a property of the signal-and-threshold design (frequent funding-rank crossings at the configured entry level), not a silent loop bug.

8. Computational details

Replication materials live under `replication/`. The canonical SPA entry point is a single paper-grade explore script (no reduced-iteration fast path):

```
# from repo root
python scripts/reproduce.py
# or: make reproduce
# smoke: make reproduce-one EXP=vol_compression_v1
# Claim 1 micro-demo (synthetic seal; seconds):
python scripts/reproduce_seal_demo.py
# or: make reproduce-seal
```

`scripts/reproduce.py` runs `run_explore()` only: registered-experiment holdouts are not

broken. Irreversible strategy commit remains `run_commit()` outside the default SPA command. Claim 1 is checked by `scripts/reproduce_seal_demo.py`, which uses a temporary seal file and writes `replication/output_seal_demo/` (see Subsection 5.1). Paper-grade SPA uses fixed `n_spa_iters = 2000`; there is no reduced-iteration fast script. Smoke tests select fewer strategies at the same SPA precision (`-strategies vol_compression_v1` or `make reproduce-one EXP=...`).

Global seed is 42. The committed cross-check sample under the `replication/` tree records package version `0.5.1`, capture time, git commit, per-strategy metrics, and wall times on Windows AMD64 (Python 3.14.4); the directory name is `output_paper_grade`. Measured times for that sample were approximately 53 minutes for all three strategies (pullback ∼6 min, volatility compression ∼12 min, funding-rate carry ∼36 min). Cold Numba caches or slower hardware can push the full pipeline toward one to two hours. JSS-style "about one hour on a regular PC" is met on the measured box.

Install the public distribution from the Python Package Index as `pip install sealedge==0.5.1` (import path remains **quant_lib**), or clone https://github.com/Hansiongs/sealedge and install editable from the repository root. Package release `0.5.1` is tagged `v0.5.1` and archived on Zenodo (https://doi.org/10.5281/zenodo.21329428). The manuscript PDF, LaTeX sources under `paper/`, and replication docs track the git commit used for submission (tag `v0.5.2` when present; otherwise name the commit in the cover letter). Platform dependencies for reproduction include `numpy`, `pandas`, `scipy`, `numba`, and `optuna` (see `replication/README.md`). Seal operations require `QUANT_LIB_HMAC_SECRET`. The reproduction script may auto-set a development default for explore-only runs; that default is not a production secret-management policy and should be replaced for any shared or long-lived deployment. Large `data_cache/` OHLCV files are external to the manuscript bundle: reviewers must pre-cache `BTCUSDT`, `ETHUSDT`, and `SOLUSDT` 1h bars (and funding series where the experiment needs them) with history before `train_start = 2021-07-01`, including volume lookback and age floors used by universe filters. `scripts/reproduce.py` fails pre-flight when required cache files or date coverage are missing; it does not fetch market data automatically. File names, minimum history relative to `train_start = 2021-07-01`, funding masters for `funding_rate_carry`, and the public `quant_lib.tools.data.fetch_klines` / `fetch_funding` helpers are documented in `replication/README.md` and `replication/data_manifest.md`.

Linux fresh-install verification of the package test suite (including optional visualization extras for plotting tests) is documented in repository helper scripts under `scripts/_linux_*.sh`. Sample git commit hashes in `results.json` refer to the environment that produced the sample; the frozen JSS source snapshot at submission may differ and should be named in the cover materials. Source and the importable package are distributed under the GNU General Public License, version 3 or later (GPL-3.0-or-later), matching the repository `LICENSE` and package metadata at version `0.5.1`.

9. Summary

sealedge turns sealed-holdout evaluation and multiple-testing practice into package defaults and a public explore/commit API: HMAC-SHA256 holdout seals, PSR and SPA on engine return objects, and deterministic walk-forward analysis. The public name is **sealedge**; the

import remains **quant_lib**. The underlying methods are standard (Bailey and de Prado 2012, 2014; Hansen 2005; White 2000; Pardo 2008; Benjamini and Hochberg 1995); what the package adds is the wiring and the replication artifacts.

At `n_spa_iters = 2000`, the paper-grade explore sample yields SPA p-values of 1.0000 for `vol_compression_v1` (438 trades), approximately 0.9300 for `pullback_sniper_rsi` (180 trades), and approximately 0.0005 for `funding_rate_carry` (3446 trades) (Section 7). These figures illustrate package outputs under a fixed seed and budget; they are not strategy recommendations. Holdout PSR is not part of the explore sample; it is available only after commit.

Limitations. Paper-grade SPA is computationally heavy: the three-strategy sample totals about 53 minutes on the reported host, so interactive work may use fewer iterations only outside the frozen manuscript path. SPA, PSR, FDR, and related diagnostics answer statistical nulls or multiplicity adjustments under stated constructions; they do not certify economic capacity, latency, or live fill quality. Dual names (**sealedge** / **quant_lib**) preserve import stability but add a documentation burden. The JSS artifact is frozen at submission; later API and experiment changes live in the open repository. Comparison cells record product surfaces observed at draft time; similar hygiene can be built on other engines by users, which the matrix marks as "user".

Outlook. Replication materials under `replication/`, package sources, and the docstring / **mkdocs** help surfaces (Section 4.1) support re-running the explore path without opening the holdout seal. Future work is maintenance and additional registered experiments in the repository rather than changes to the archival manuscript numbers.

Acknowledgments

This manuscript was drafted by the author. A large language model was used to assist in language editing, clarity improvement, and text organization. All code, results, and prose were reviewed and verified by the author prior to submission.

References

Akiba T, Sano S, Yanase T, Ohta T, Koyama M (2019). "**Optuna**: A Next-generation Hyperparameter Optimization Framework." *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 2623–2631. doi:10.1145/3292500.3330701.

Bailey DH, de Prado ML (2012). "The Sharpe Ratio Efficient Frontier." *Journal of Risk*, **15**(2), 3–44.

Bailey DH, de Prado ML (2014). "The Deflated Sharpe Ratio: Correcting for Selection Bias, Backtest Overfitting, and Non-Normality." *Journal of Portfolio Management*, **40**(5), 94–107.

Benjamini Y, Hochberg Y (1995). "Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing." *Journal of the Royal Statistical Society, Series B*, **57**(1), 289–300.

Hansen PR (2005). "A Test for Superior Predictive Ability." *Journal of Business & Economic Statistics*, **23**(4), 365–380. doi:10.1198/073500105000000063.

Kish L (1965). *Survey Sampling*. John Wiley & Sons, New York.

Krawczyk H, Bellare M, Canetti R (1997). "HMAC: Keyed-Hashing for Message Authentication." *RFC 2104*, Internet Engineering Task Force. doi:10.17487/RFC2104. URL https://www.rfc-editor.org/rfc/rfc2104.

Lam SK, Pitrou A, Seibert S (2015). "**Numba**: A LLVM-Based Python JIT Compiler." *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, pp. 1–6. doi:10.1145/2833157.2833162.

Pardo R (2008). *The Evaluation and Optimization of Trading Strategies*. 2nd edition. John Wiley & Sons, Hoboken.

Phipson B, Smyth GK (2010). "Permutation *P*-Values Should Never Be Zero: Calculating Exact *P*-Values When Permutations Are Randomly Drawn." *Statistical Applications in Genetics and Molecular Biology*, **9**(1), Article 39. doi:10.2202/1544-6115.1585.

Polakow O (2024). ***vectorbt**: Vectorized Backtesting in Python*. Software documentation, URL https://vectorbt.dev/.

Politis DN, Romano JP (1994). "The Stationary Bootstrap." *Journal of the American Statistical Association*, **89**(428), 1303–1313. doi:10.1080/01621459.1994.10476870.

Python Software Foundation (2024). *Python: A Dynamic, Open Source Programming Language*. URL https://www.python.org/.

QuantConnect (2024). ***Lean**: Open Source Algorithmic Trading Engine*. Software documentation, URL https://www.lean.io/.

Quantopian Inc (2020). ***Zipline**: A Python Algorithmic Trading Library*. Software documentation, URL https://github.com/quantopian/zipline.

Rodriguez D (2024). ***backtrader**: Python Backtesting Library*. Software documentation, URL https://www.backtrader.com/.

White H (2000). "A Reality Check for Data Snooping." *Econometrica*, **68**(5), 1097–1126.

Winarto H (2026). ***sealedge**: Sealed Holdouts and Multiple-Testing Defaults for Crypto Backtests in Python*. Python package version 0.5.1, URL https://github.com/Hansiongs/sealedge.

Affiliation:

Hansen Winarto
Information Systems Department, BINUS Online Learning,
Bina Nusantara University,
Jakarta, Indonesia 11480
E-mail: hansenwinarto@binus.ac.id
URL: https://github.com/Hansiongs/sealedge

Journal of Statistical Software https://www.jstatsoft.org/
published by the Foundation for Open Access Statistics https://www.foastat.org/
MMMMMM YYYY, Volume VV, Issue II *Submitted:* yyyy-mm-dd
doi:10.18637/jss.v000.i00 *Accepted:* yyyy-mm-dd